WEEKS 16–17 · LABORATORY ACTIVITY

Make It Fast & Fair, Then Ship It

Accessibility + performance, then an automated pipeline — on the SecureNotes app

Course	Units	Work in	Est. time	Submit
Web Systems and Technologies	1 (Laboratory)	Pairs	3–4 hours	GitHub PR links

The scenario. SecureNotes is built, secured (Weeks 13–14), and tested (Week 15). The API works and it's safe. But the page people actually use is **inaccessible and slow**, and nothing stops a teammate from quietly breaking it later. This lab is the last mile: first make the front end **fast and usable for everyone** (Week 16), then build the **pipeline** that runs every check automatically on each push (Week 17), and ship it. Part A fixes it; Part B makes sure it *stays* fixed.

What you'll practice

Part A — fast & fair (Week 16)

- WCAG basics: lang, titles, alt text
- landmarks, heading order, real buttons
- form labels tied to inputs
- color contrast that meets AA (4.5:1)
- perf: deferred JS, image dimensions, lazy loading

Part B — ship it (Week 17)

- a GitHub Actions workflow file
- triggers on push & pull_request
- a job on a clean Ubuntu runner
- steps: checkout → Python → install → test
- a red pipeline that blocks a bad merge

Setup (10 minutes)

```
# inside the student folder
python -m venv venv
source venv/bin/activate   # Win:
venv\Scripts\activate
pip install -r requirements.txt

uvicorn app.main:app --reload   # the app
# open http://127.0.0.1:8000/app/
pytest -q                  # 12 app tests pass;
many checks red
```

The front end is served at `/app/`. The API docs are at `/docs`.

Accounts

- `admin / admin123` (admin)
- `alice / alicepass` — owns note 1
- `bob / bobpass` — owns note 2

You edit only three files: `app/static/index.html`, `app/static/styles.css`, and `.github/workflows/ci.yml`. Don't touch `app/main.py` or the tests.

Keep the bcrypt pin (`bcrypt==4.0.1`).

Part A — Make it accessible & fast (~90 min)

Edit inside the `>>> TASK n ... <<< END TASK n` markers in `index.html` and `styles.css`. Check your work with `pytest tests/test_accessibility.py tests/test_performance.py`.

Task	Where	What to do	Why it matters
1	index.html	**Perceivable.** Add `lang` to `<html>`, a real `<title>`, and `alt` on both images.	Screen readers announce the language, the tab, and what each image shows.
2	index.html		

		Structure & controls. Make the content a `<main>`, fix headings so they don't skip (h1→h2), and turn the fake `<div class="btn">` into a real `<button>`.	Keyboard users can reach controls; everyone can navigate by landmark and heading.
3	index.html	**Labels.** Replace the fake labels with real `<label for="...">` tied to each input `id`.	A screen reader reads the field name when focus lands on the box.
4	styles.css	**Contrast.** Darken `--ink` and `--brand` until both contrast tests pass (AA = 4.5:1).	Low-contrast text is unreadable in sunlight or with low vision. The test does the math.
5	index.html	**Performance.** `defer` the script, add `width` / `height` to both images, and `loading="lazy"` to the content image.	The page paints sooner and stops jumping around as images load (CLS).

> **Tests catch the mechanical stuff — you catch the rest.** A test can see a missing `alt`, but not whether your alt text is *useful*. After the suite is green, tab through the page with your keyboard only, and run **Lighthouse** (Chrome DevTools → Lighthouse → Accessibility + Performance) to see the full score.

Part B — Ship it on a pipeline (~45 min)

Build `.github/workflows/ci.yml` so every push and PR runs the **whole** suite — the security tests *and* your new Week 16 checks — on a clean machine. Check with `pytest tests/test_pipeline.py`.

Task	What to add
6	**Triggers.** Run `on:` a `push` and a `pull_request` to `main`.
7	**The job.** One `test` job, `runs-on: ubuntu-latest`, with four steps: `actions/checkout` → `actions/setup-python` → `pip install -r requirements.txt` → `pytest`.

```
# the shape of a step
- name: Run the full test suite
  run: pytest -q
```

Everything here was in the Week 17 lecture — workflow, trigger, job, step. The pipeline test even accepts YAML's quirk of reading a bare `on:` as `True`, so don't worry if you see that in Python.

Part C — Push & peer review (~30 min)

```
git init
git add . && git commit -m "Starter: SecureNotes front end + empty pipeline"
git branch -M main
git remote add origin https://github.com/<you>/securenotes-shipit.git
git push -u origin main
git checkout -b fast-and-fair
# ...do Part A, then Part B...
git add . && git commit -m "Accessible + fast front end, CI pipeline green"
git push -u origin fast-and-fair
```

Open a pull request (`fast-and-fair → main`) and watch the **Actions** tab — your pipeline runs and should end in a green check. Then swap PRs with another pair: pull their branch, run `pytest`, and leave **line comments** on the diff before you **Approve** or **Request changes**. A red pipeline means the merge button stays locked — that's the point.

Deliverables & rubric

Submit (via the LMS):

1. GitHub **repo** link (`fast-and-fair` branch).
2. Your **pull request** link, with the green Actions check.
3. The **PR you reviewed** (line comments + verdict).
4. Completed `TASKS.md` (checklist + reflection).

This is the last lab before the final — it ties together everything from Week 1 to Week 17.

Criteria	Pts
Part A — accessibility (tasks 1–4)	**45**
Part A — performance (task 5)	**15**
Part B — pipeline green (local + GitHub)	**25**
Part C — PR + peer review	**10**
Reflection answered	**5**